COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	August 7, 2026	**DATE SUBMITTED:**	August 26, 2026
NAME:	Uchi, Mao Sophie S.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5.1 **Queues**

1. PROCEDURE OUTPUT

ILO A: Create queue using C++ STL

Code

ilo_a.cpp

```cpp
//C++ code to illustrate Queue in the Standard Template Library (STL)
#include <iostream>
#include <queue>

void display(std::queue<int> q)
{
    std::queue<int> c=q;

    while (!c.empty())
    {
        std::cout << " " <<c.front();
        c.pop();
    }
    std::cout << "\n";
}

int main(){

    std::queue<int> a;

    // Insert elements into the queue
    a.push(10);
    a.push(20);
    a.push(30);

    std::cout << "The queue a is";
    display(a);

```

```
29      std::cout << "a.empty() :" <<a.empty() << "\n";
30      std::cout << "a.size()  :" << a.size() << "\n";
31      std::cout << "a.front() :" << a.front() << "\n";
32      std::cout << "a.back()  :" << a.back()<< "\n";
33
34      std::cout << "After a.pop() : ";
35      a.pop();
36      display(a);
37
38      a.push(40);
39
40      std::cout << "The queue a is : ";
41      display(a);
42
43      return 0;
44  }
```

Output

```
The queue a is 10 20 30
a.empty() :0
a.size()  :3
a.front() :10
a.back()  :30
After a.pop() :  20 30
The queue a is :  20 30 40
PS D:\Downloads (HDD)\CPE010\HOA5\output>
```

ILO B.1: Linked List Implementation

Codes

main.cpp

main.cpp > main()

```cpp
#include <iostream>
#include "queue.h"
#include "queueArr.h"
#include <queue>

int main() {

    qNode<int>* front = nullptr;
    qNode<int>* back = nullptr;

    enqueue(1, &front, &back);
    std::cout << "front: " << front->data << " back: " << back->data << "\n";
    enqueue(2, &front, &back);
    enqueue(3, &front, &back);
    std::cout << "front: " << front->data << " back: " << back->data << "\n";

    display(front);
    dequeue(&front, &back);
    std::cout << "front: " << front->data << " back: " << back->data << "\n";
    dequeue(&front, &back);
    std::cout << "front: " << front->data << " back: " << back->data << "\n";
    dequeue(&front, &back);
    dequeue(&front, &back);
    display(front);

    return 0;
}
```

queue.h

```
C queue.h > dequeue<T>(qNode<T>**, qNode<T>**)
1   #ifndef QUEUELL_H
2   #define QUEUELL_H
3   #include <iostream>
4
5   template <typename T>
6   class qNode{
7   public:
8       T data;
9       qNode<T> *next;
10  };
11
12  //creating the new node
13  template <typename T>
14  qNode<T> *new_node(T newData){
15      qNode<T> *newNode = new qNode<T>;
16      newNode->data = newData;
17      newNode->next = NULL;
18      return newNode;
19  }
20
21  template <typename T>
22  void enqueue(T newData, qNode<T>** frontptr, qNode<T>** backptr){
23      //create a new node
24      qNode<T>* newNode = new_node(newData);
25
26      //inserting into an empty queue
27
28      //how do we know if the queue is empty?
29      if ((*frontptr) == nullptr && (*backptr) == nullptr){
30          //assign the frontptr to the newNode
31          (*frontptr) = newNode;
32          //assign the backptr to the newNode
33          (*backptr) = newNode;
34      }
35
36      //point the back next to the newNode
37      (*backptr)->next = newNode;
38      //the new node will be the new backptr
39      (*backptr) = newNode;
40      return;
41  }
```

```
42
43  template <typename T>
44  void dequeue(qNode<T>** frontptr, qNode<T>** backptr){
45      //how to determine if the queue is empty?
46      if ((*frontptr) == nullptr && (*backptr) == nullptr){
47          std::cout << "The queue is empty. Nothing to dequeue.\n";
48          return;
49      }
50
51      // creating a temporary pointer:
52      qNode<T>* tempPtr = nullptr;
53      //store the frontptr to the tempPtr
54      tempPtr = (*frontptr);
55
56      //check if queue has only 1 node left??
57      if ((*frontptr) == (*backptr)){
58          (*frontptr) = nullptr;
59          (*backptr) = nullptr;
60          delete tempPtr;
61          return;
62      }
63
64      //update the frontptr
65      (*frontptr) = (*frontptr)->next;
66      // point the tempPtr -> next to null
67      tempPtr->next = nullptr;
68      delete tempPtr;
69
70  }
```

```
71
72  template <typename T>
73  void display(qNode<T>* frontPtr){
74      //catch if the queue is empty
75      if(frontPtr == nullptr) {
76          std::cout << "Queue Empty\n";
77          return;
78      }
79      //temp pointer to store the front ptr
80      qNode<T>* current = frontPtr;
```

```
81
82      //create a loop that will print the tempPtr->data
83      while(current != nullptr){
84          std::cout << current->data << " ";
85          current = current->next;
86      }
87      std::cout << std::endl;
88      return;
89  }
90
91  #endif
```

Output

```
front: 1 back: 1
front: 1 back: 3
1 2 3
front: 2 back: 3
front: 3 back: 3
The queue is empty. Nothing to dequeue.
Queue Empty
PS D:\Downloads (HDD)\CPE010\HOA5>
```

ILO B.2. Array-Based Implementation

Codes

main.cpp

```cpp
#include <iostream>
#include "queue.h"
#include "queueArr.h"
#include <queue>

int main() {

    //object creation
    queueArr<char> q(6);

    q.Enqueue(1);
    std::cout << "front: " << q.Front() << " Back: " <<q.Back() << "Size: " << q.Size() << std::endl;
    q.Enqueue(2);
    std::cout << "front: " << q.Front() << " Back: " <<q.Back() << "Size: " << q.Size() << std::endl;
    q.Enqueue(3);
    std::cout << "front: " << q.Front() << " Back: " <<q.Back() << "Size: " << q.Size() << std::endl;
    q.Enqueue(4);
    std::cout << "front: " << q.Front() << " Back: " <<q.Back() << "Size: " << q.Size() << std::endl;
    q.Enqueue(5);
    std::cout << "front: " << q.Front() << " Back: " <<q.Back() << "Size: " << q.Size() << std::endl;
    q.Enqueue(6);
    std::cout << "front: " << q.Front() << " Back: " <<q.Back() << "Size: " << q.Size() << std::endl;
    q.Enqueue(7);
    q.Dequeue();
    std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;
    q.Dequeue();
    q.Dequeue();
    q.Dequeue();
    q.Dequeue();
    q.Dequeue();
    std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;

    return 0;
}
```

queueArr.h

```cpp
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>
```

```
4
5   template <typename T>
6   class queueArr{
7       private:
8           //pointer to dynamically allocated array
9           T* q_array;
10          size_t q_capacity;
11          size_t q_size;
12          int q_front;
13          int q_back;
14
15      public:
16          //constuctor
17          queueArr(size_t q_capacity);
18
19          bool Empty();
20          size_t Size();
21          void Clear();
22
23          T Front();
24          T Back();
25
26          void Enqueue(T value);
27          void Dequeue();
28
29          // copy constructor:
30          //queueArr(const queueArr& other);
31
32          // copy assignment operator
33          //queueArr& operator=(const queueArr& other);
34
35          //destructor
36          ~queueArr();
```

```
37
38  };
39
40  //constructor
41  template <typename T>
42  queueArr<T>::queueArr(size_t capacity){
43      q_capacity = capacity;
44      q_size = 0;
45      q_front = 0;
46      q_back = -1;
47      q_array = new T[q_capacity];
48  }
49
50  template<typename T>
51  bool queueArr<T>::Empty(){
52      return q_size == 0;
53  }
54
55  template<typename T>
56  size_t queueArr<T>::Size(){
57      return q_size;
58  }
59
60  template<typename T>
61  void queueArr<T>::Clear(){
62      q_size = 0;
63      q_front = 0;
64      q_back = -1;
65  }
66
```

```cpp
template<typename T>
T queueArr<T>::Front(){
    //what will happen if the queue is empty?
    if(Empty()){
        std::cout << "The Queue is Empty. \n";
        return T{};
    }

    return q_array[q_front];
}

//back
template<typename T>
T queueArr<T>::Back(){
    //what will happen if the queue is empty?
    if(Empty()){
        std::cout << "The Queue is Empty. \n";
        return T{};
    }

    return q_array[q_back];
}

template<typename T>
void queueArr<T>::Enqueue(T value){
    // check if the queue is full
    if(q_size == q_capacity){
        std::cout << "Queue is Full\n";
        return;
    }
    // compute for the q_back for circular
    q_back = (q_back + 1) % q_capacity;

```

```
100     //assign the q_array with the updated q_back to the value
101     q_array[q_back] = value;
102 
103     //incrementing q_size
104     q_size++;
105 
106     return;
107 }
108 
109 template<typename T>
110 void queueArr<T>::Dequeue(){
111     // check if the queue is empty
112     if(Empty()){
113         std::cout << "Queue is Empty\n";
114         return;
115     }
116     // move q_front circularly
117     q_front = (q_front + 1) % q_capacity;
118 
119     //decrementing q_size
120     q_size--;
121 
122     //reset index when queue becomes empty
123     if(Empty()){
124     Clear();
125     return;
126     }
127 }
128 
129 template <typename T>
130 queueArr<T>::~queueArr() {
131     delete[] q_array;
132 }
```

```
100     //assign the q_array with the updated q_back to the value
101     q_array[q_back] = value;
102 
103     //incrementing q_size
104     q_size++;
105 
106     return;
107 }
108 
109 template<typename T>
110 void queueArr<T>::Dequeue(){
111     // check if the queue is empty
112     if(Empty()){
113         std::cout << "Queue is Empty\n";
114         return;
115     }
116     // move q_front circularly
117     q_front = (q_front + 1) % q_capacity;
118 
119     //decrementing q_size
120     q_size--;
121 
122     //reset index when queue becomes empty
123     if(Empty()){
124     Clear();
125     return;
126     }
127 }
128 
129 template <typename T>
130 queueArr<T>::~queueArr() {
131     delete[] q_array;
132 }
133 
134 #endif // QUEUEARR_H
```

Output

```
front:  Back: Size: 1
front:  Back: Size: 2
front:  Back: Size: 3
front:  Back: Size: 4
front:  Back: Size: 5
front:  Back: Size: 6
Queue is Full
Front:  Back:  Size: 5
Front: The Queue is Empty.
 Back: The Queue is Empty.
 Size: 0
PS D:\Downloads (HDD)\CPE010\HOA5\output>
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using queue implementation

Create a class called Job (comprising an ID for the job, the name of the user who submitted it, and the number of pages)

```
6   class Job
7   {
8   public:
9       int userID;
10      std::string userName;
11      int num_pages;
12  };
```

```
Input your choice: 1
Job ID: 2
Username: Johnny
Number of Pages: 2

Job had been added.
```

Create a class called Printer. This will provide an interface to add new jobs and process all the jobs added so far.

```
14  class Printer
15  {
16  private:
17      Job *q_array;
18      size_t q_capacity;
19      size_t q_size;
20      int q_front, q_back;
21
22  public:
23      //Constructor
24      Printer(size_t capacity = 10);
25      //Methods
26      bool isEmpty() const;
27      bool isFull() const;
28      void Clear();
29      void addedJob(Job value);
30      Job deleteJob();
31      void DisplayJob();
32      void JobProcess();
33      //Destructor
34      ~Printer();
35  };
```

```
--- OFFICE PRINTER ---
1. Add Job
2. Process Job/s
3. Remove Job
4. Display Job/s
5. Clear Job/s
6. Exit Program
```

To implement the printer class, it will need to store all the pending jobs. We'll implement a very basic strategy – first come, first served. Whoever submits the job first will be the first to get the job done

```
--- OFFICE PRINTER ---
1. Add Job
2. Process Job/s
3. Remove Job
4. Display Job/s
5. Clear Job/s
6. Exit Program

Input your choice: 4
ID: 1, Username: Josh, Pages: 5
ID: 2, Username: Johnny, Pages: 2
ID: 3, Username: Jack, Pages: 2
```

Finally, simulate a scenario where multiple people are adding jobs to the printer, and the printer is processing them one by one.

```
--- OFFICE PRINTER ---
1. Add Job
2. Process Job/s
3. Remove Job
4. Display Job/s
5. Clear Job/s
6. Exit Program

Input your choice: 2
Printing Job ID: 1, Username: Jim, Pages: 3
Printing Job ID: 2, Username: Jimmy, Pages: 3
Printing Job ID: 3, Username: John, Pages: 3
Job has been Printed.
```

Defend your choice of internal representation: Why did you use arrays or linked lists?

Linked List handles the size better than array as array needs declaration for fixed size to create a list, also when the fixed size is full, you'll have to create another size for the larger data, while Linked list doesn't have a fixed size and will create or delete a node, is when the “size” will increase/decrease. With array, It is much harder to add or delete an element as the add or delete operators update sequentially and takes longer to debug, while Linked List can insert or remove an element whichever in the list, not needing to manually move the other elements as it will automatically updates the new size and placement.

Output Analysis

- Provide the output after performing each task above.
- Include your analysis: focus on why you think the output is the way it is.

As we create a program for the Office Printer for employees who are using the same printer using a linked list. From this program, we’ve implemented a switch case menu for the Printer Program options, which are adding job for the printer, processing the jobs that was added, removing the jobs that was added, displaying the jobs that was added, clearing all the jobs that was added and lastly, exiting the Printing Program. For adding a job, we will input the user’s ID, their username and pages that are needed to print; these are already the new node itself, but as a bundle, it will be considered as 1 node even if we implemented more than 1 user-input as long as they’re allocated together. For processing a job, but what was done here is every additional job (new node) in the processing option will stay there until it is ready to print and will be deleted from the list.

Conclusion
For my conclusion, I have learned on how to implement a queue operation as this is similar to Linked List and Stacking, but for queue, you'll need the 1st element or head in the array to enter first and also first to go out, while the last element or rear to enter last and also last to leave the array. These are one of the last topics that was taught to us this semester as we are near approaching midterms, but even so, I'm already struggling to some of these supplementary activities has been progressively getting difficult each laboratories as I got help from different people, which I am grateful for, yet there are times I do not know what I'm doing at all, but I really try to catch up and understand every new operations and iterations.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Quillbot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Summarization**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Summary of my own uncertain understanding**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.